Files

Definition

A **file system** is a key concept in almost all operating systems. Its main job is to hide the hardware details of disks and other I/O devices and present the user with a clean, device-independent way of storing and accessing data as **files**. Based on user text :contentReference[oaicite:0]index=0

1. Basic File Operations

Common System Calls

The operating system provides system calls to:

- create a file,
- remove a file,
- read a file,
- write a file,
- open a file,
- close a file.

Before a file is read or written, it must usually be **opened**. After use, it should be **closed**.

2. Directories

Directory

A **directory** is a way of grouping related files together. Operating systems use directories to organize files in a structured manner.

Why Directories are Useful

A user may keep:

- one directory for course materials,
- one directory for email,
- one directory for web pages,
- one directory for personal documents.

Directory Operations

The operating system provides system calls to:

- create directories,
- remove directories,
- place files inside directories,
- remove files from directories.

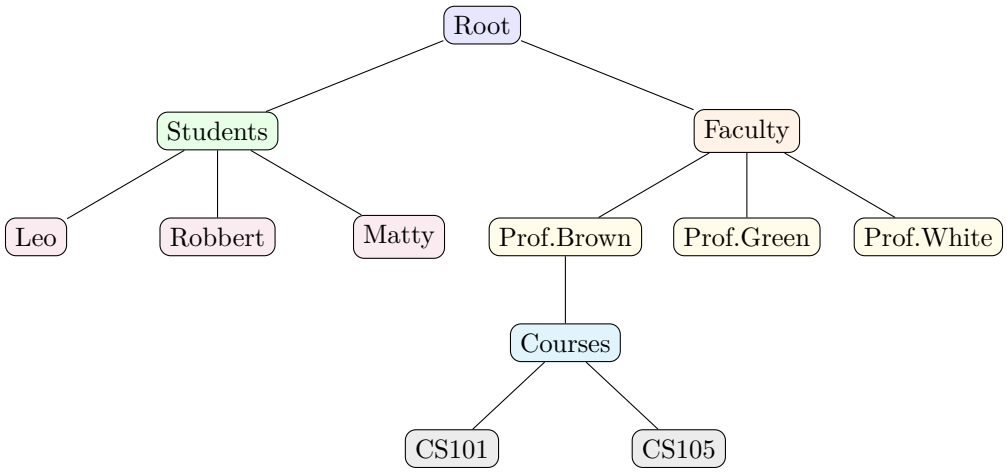
Directory entries may be either **files** or **other directories**.

3. File-System Hierarchy

Tree Structure

Directories and files form a **hierarchy**, often organized as a tree. At the top is the **root directory**.

3.1 Simple File Hierarchy Diagram



Note

File-system hierarchies are usually much deeper and longer-lived than process hierarchies. A directory tree may remain in use for years.

4. Path Names

Absolute Path Name

Every file can be identified by giving its **path name** from the root directory. Such a path is called an **absolute path name**.

Example

An example of an absolute UNIX path is:

$$/Faculty/Prof.Brown/Courses/CS101$$

The leading slash "/" means the path starts from the root directory.

UNIX vs Windows

- In **UNIX**, the separator is `/`
- In **Windows**, the separator is `\`

So the same path in Windows style would be:

```
\Faculty\Prof.Brown\Courses\CS101
```

5. Current Working Directory

Working Directory

Each process has a **current working directory**. If a path does not begin with a slash, the operating system searches for it relative to the current working directory.

Example

If the current working directory is:

$$/Faculty/Prof.Brown$$

then the relative path

$$Courses/CS101$$

refers to the same file as the absolute path:

$$/Faculty/Prof.Brown/Courses/CS101$$

Changing Directory

A process can change its working directory by using a system call.

6. Opening Files and File Descriptors

Open and Permissions

Before a file can be read or written, it must be **opened**. At that time, the operating system checks whether access is allowed.

File Descriptor

If access is allowed, the operating system returns a small integer called a **file descriptor**. This file descriptor is then used in later operations like reading and writing.

Access Failure

If permission is denied, the operating system returns an **error code**.

7. Mounted File System

Mounted File System

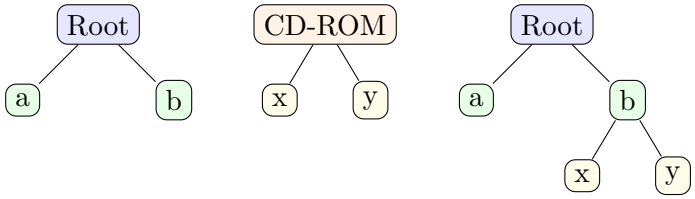
A **mounted file system** allows a separate file system, such as one on a CD-ROM, USB drive, or external disk, to be attached to the main directory tree.

Main Idea

Instead of using device names in file paths, UNIX attaches the external file system to an existing directory in the main tree. This is called **mounting**.

7.1 Simple Mount Diagram

(a) Before Mounting **(b) After Mounting**



Example

If a CD-ROM file system is mounted on directory `/b`, then its files may be accessed as:

$$/b/x \quad \text{and} \quad /b/y$$

8. Special Files

Special File

UNIX uses **special files** to make I/O devices appear like files. This allows devices to be accessed using ordinary file operations such as read and write.

Types of Special Files

There are two main types:

- **Block special files**
- **Character special files**

Block vs Character

- **Block special files** represent devices with randomly addressable blocks, such as disks.
- **Character special files** represent devices that handle character streams, such as printers and modems.

Example

By convention, special files are stored in the `/dev` directory.
Example:

$$/dev/lp$$

may represent a printer.

9. Pipes

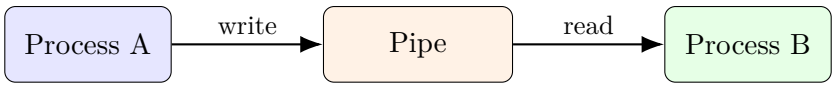
Pipe

A **pipe** is a pseudofile used to connect two processes. It allows one process to send data to another in a file-like manner.

How a Pipe Works

- Process A writes data to the pipe.
- Process B reads data from the pipe.
- The interaction looks similar to file writing and file reading.

9.1 Pipe Diagram



Key Idea

In UNIX, communication through pipes looks very similar to ordinary file I/O. A process may not even know whether it is writing to a real file or to a pipe unless it checks specially.